ALU Microarchitecture

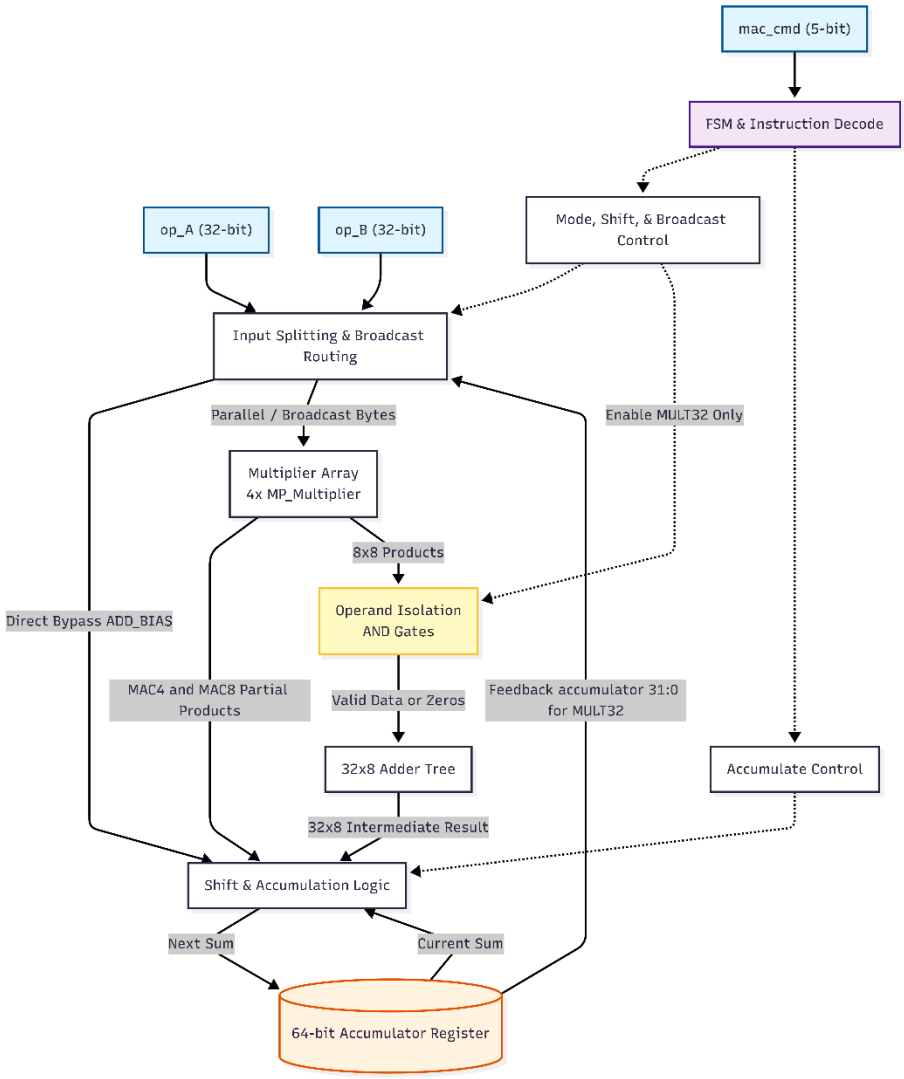

Figure 1. Block-level microarchitecture.

High-Level Overview

The ALU module acts as the primary arithmetic core of the PE. It takes two 32-bit operands and computes various multiply-accumulate operations based on the 5-bit controller instruction.

Interface

Port Name	Direction	Width	Description
clk	Input	1 bit	Main system clock. All accumulations and FSM state transitions trigger on positive edge.
rst	Input	1 bit	Active-low asynchronous reset. Clears the FSM state, busy/done flags, and the 64-bit accumulator.
alu_cmd	Input	5-bit bus	Driven directly by the alu_cmd output of the PE_Controller module. Dictates the current arithmetic operation: NOP (0), RST_ACC (1), MAC4 (2), MAC8 (3), ADD_BIAS (4), or MULT32 (5).
op_A	Input	32-bit bus	The first data operand. In MAC modes, it is split into four 8-bit or eight 4-bit values. In MULT32 mode, this external pin is ignored, and the datapath instead samples the lower 32 bits of the internal accumulator.
op_B	Input	32-bit bus	The second data operand. In MAC modes, it is split to match op_A. In MULT32 mode, it acts as an unsigned 32-bit scaling factor, and its bytes are broadcast sequentially. In ADD_BIAS mode, it loads a 32-bit bias and adds it directly into the accumulator.
busy	Output	1 bit	Status flag indicating the module is currently executing a multi-cycle operation (specifically, MULT32) and cannot accept new commands.
done	Output	1 bit	Status flag that pulses high for one cycle when a multi-cycle operation successfully finishes.
accumulator	Output	64-bit bus	The internal 64-bit register holding the running sum of all operations.

Core Datapath & Theory of Operation

The architecture is built around a flexible datapath that reuses the four underlying MP_Multiplier blocks to perform a 32x32 multiplication.

Input Routing & Feedback Loop

The 32-bit operands are split into four independent 8-bit chunks.

During standard MAC operations, the external op_A pins route directly to the 'a' ports of the four multipliers.

During a MULT32 operation, the datapath must prevent the changing accumulator from corrupting the multiplicand. To solve this, a 32-bit holding register captures a snapshot of the lower 32 bits of the accumulator on the first cycle. An internal multiplexer then routes this latched value to the 'a' ports for the duration of the multi-cycle calculation.

Dynamic Broadcast

The routing for op_B is controlled by the use_broadcast signal.

- When low (parallel mode), the four bytes of op_B feed directly into the b ports.
- When high (broadcast mode), a single byte from op_B is broadcast simultaneously to all four multipliers.

Sign Control

To execute operations properly, the module dynamically manipulates the sign flags of the multipliers.

- **'A' Ports:** During standard MAC operations, all four bytes of op_A are treated as signed values. However, during a MULT32 multi-cycle sequence, the 32-bit internal accumulator is sliced into bytes. To preserve the math, the lower three bytes are forced to be unsigned, while only the Most Significant Byte (byteA3) retains its signed property.
- **'B' Ports:** The 'b' ports are treated as signed during MAC operations, but are forced to be unsigned during a MULT32 operation so the hardware correctly interprets the 32-bit sequence as a scaling factor.

Accumulation and Adder Trees

While standard MAC operations sum the multiplier outputs directly into the 64-bit accumulator, the MULT32 operation requires an intermediate adder tree to stitch the four 8x8 products into a single 32x8 result.

Mathematically, multiplying a 32-bit A by an 8-bit B_{byte} expands to:

$$A \times B_{byte} = (A_3 \cdot B_{byte}) \cdot 2^{24} + (A_2 \cdot B_{byte}) \cdot 2^{16} + (A_1 \cdot B_{byte}) \cdot 2^{8} + (A_0 \cdot B_{byte})$$

The hardware mirrors this equation using wire concatenation to perform the mathematical shifts (multiplying by powers of 2):

- **First Stage (24-bit Adders):** adder24_1 shifts the second byte's product left by 8 bits and adds it to the first byte's product. adder24_2 groups the top two bytes using the exact same logic.
- **Final Stage (40-bit Adder):** The module combines the two 24-bit chunks. adder24_2 is shifted left by 16 bits and added to the zero-padded adder24_1. This combinatorial sum (adder40) constructs the exact partial product of the full 32-bit op_A multiplied by a single byte of op_B.
- **Operand Isolation:** A layer of AND gates is inserted between the multiplier array and the 32x8 intermediate adder tree. When alu_cmd is set to MULT32, these gates pass the 8x8 products through to the adders. For all other instructions, the gates force the adder tree inputs to zero. This prevents the constantly toggling multiplier outputs from propagating down the datapath, saving dynamic power.
- **Direct Accumulator Bypass:** To support efficient bias loading, the datapath includes a bypass route. During an ADD_BIAS instruction, the 32-bit op_B input bypasses the multipliers and the 40-bit adder tree entirely. It is sign-extended to 64 bits and added directly to the current accumulator value.

Modes of Operation (FSM)

The module is driven by a 5-bit controller input that defines the active instruction. The operations are divided into single-cycle parallel instructions and a multi-cycle scalar instruction managed by an internal FSM.

Single-Cycle Operations

When the module is not busy, it can execute the following commands in one clock cycle:

- **NOP (5'd0):** No operation. The datapath remains idle, and the 64-bit accumulator holds its current state. The PE_Controller issues this safe command when intercepting instructions meant for other downstream modules.
- **RST_ACC (5'd1):** Synchronously clears the 64-bit accumulator to zero.
- **MAC4 (5'd2):** Configures the underlying blocks in 4x4 mode. The eight 8-bit partial products are individually sign-extended to 64 bits and summed directly into the accumulator.
- **MAC8 (5'd3):** Configures the multipliers in 8x8 mode. The four 16-bit products are sign-extended to 64 bits and summed into the accumulator.
- **ADD_BIAS (5'd4):** Bypasses the multiplier array. The 32-bit op_B input is sign-extended to 64 bits and added to the current accumulator value.

Multi-Cycle Operation: MULT32 (32x32 Multiplication)

The MULT32 command (5'd5) performs a full 32x32 multiplication to scale the accumulated result. The hardware calculates the result over four clock cycles by multiplying the lower 32 bits of the internal accumulator by one byte of the unsigned op_B scaling factor per cycle.

The module asserts the busy flag to prevent interference and steps through the sequence:

- **Cycle 0:** The FSM registers the MULT32 command and immediately latches the lower 32 bits of the current accumulator into the holding register. Simultaneously, it broadcasts the lowest byte of op_B. The 40-bit intermediate

result is sign-extended to 64 bits and overwrites the main accumulator. The FSM increments to 1.

- **Cycle 1:** The FSM broadcasts the second byte. The hardware shifts the new 40-bit result left by 8 bits, sign-extends it, and adds it to the accumulator. The FSM increments to 2.
- The FSM broadcasts the third byte. The 40-bit result is shifted left by 16 bits, sign-extended, and added to the accumulator. The FSM increments to 3.
- **Cycle 3:** The FSM broadcasts the final Most Significant Byte. The 40-bit result is shifted left by 24 bits and added to the accumulator. On this final clock edge, the FSM drops the busy flag, pulses the done flag high for one cycle, and resets the counter.

Timing, Clocking, and Reset Strategy

The module is a synchronous sequential design that manages its internal state and accumulator through clocking and reset paths.

- **Timing and Critical Path:** The expected critical path delay for the module is approximately X nanoseconds. This path is activated during the multi-cycle MULT32 instruction. The delay chains through the broadcast multiplexers, the internal 8x8 multipliers (which take up most of that delay), the operand isolation logic, the two stages of the intermediate adder tree, the shift logic, and finally the main 64-bit adder before settling at the accumulator register.
- **Clocking:** All stateful elements operate synchronously on the positive edge of the main system clock (clk). While the intermediate combinational math evaluates continuously, the final results, FSM counter updates, and status flags are only captured on this rising edge.
- **Asynchronous Hardware Reset:** The module uses an active-low, asynchronous reset pin (rst). Asserting this signal immediately halts any operation, resets the cycle counter to 0, clears the status flags, and flushes both the 64-bit accumulator and the 32-bit holding register to 0, completely independent of the clock.
- **Synchronous Software Clear:** The datapath also supports a synchronous clear via the RST_ACC instruction. This allows the controller to flush the accumulation register between different vector operations on the next positive clock edge without needing to assert the global hardware reset. This instruction strictly clears the accumulator register and does not clear or modify the status flags.